

**ENSE706 - Data Process and
Software Modelling**

Project Progress Report

**Thanh Tran
24247169**

15/3/2026

Table of Contents

1. Introduction	1
2. Organisational and system analysis	2
2.1. Overview of the selected organisation	2
2.2. Departmental structure and responsibilities	2
2.3. Problem statement	3
2.4. Stakeholders and user roles	5
2.5. System scope, boundaries, and assumptions	6
i. System scope	6
ii. System boundaries	7
iii. Assumptions	7
3. Requirements engineering	8
3.1. Requirements elicitation	8
i. Problem statement analysis	8
ii. Existing document analysis	9
iii. Observation of existing system	10
iv. Stakeholder interview	12
3.2. User and system requirements	15
i. User requirements	15
ii. System requirements	17
3.3. Functional requirements	18
3.4. Non-functional requirements	20
3.5. Traceability between stakeholders and requirements	21
4. UML requirements modelling	22
4.1. Actor	22
4.2. Use case	23
i. Actor – use case	23
ii. System – use case	24
iii. Use case descriptions	24

4.3. Use case relationships	27
i. Include relationships.....	27
ii. Extend relationships	28
iii. Generalisation relationships.....	28
4.4. Use Case Diagram	29
5. Initial structural design	30
5.1. Class diagram	30
i. Identify classes	30
ii. Identify attributes and operations	31
iii. Identify relationships	33
5.2. Mapping to C# classes	36
5.3. Anticipated design weaknesses and limitations.....	39
6. C# implementation	41
7. Conclusion	51
References	52

1. Introduction

Modern medium to large organisations rely on advanced software systems to manage operations, coordinate between departments, and support decision-making. However, these systems are often developed over time without proper design practices, leading to maintenance challenges, scalability issues, and increased long-term risks.

This project focuses on analysing, modelling, and designing a business-critical information system within a real-world organisational context. The aim is to apply rigorous engineering techniques and Unified Modelling Language (UML) to develop a clear and structured system model, which will later be translated into an object-oriented implementation using C#.

The report is structured into several stages. It begins with organisational selection and system analysis, identifying the departmental structure, stakeholders, and system scope. This is followed by requirements engineering, where functional and non-functional requirements are defined and linked to stakeholder needs. UML modelling is then used to represent system functionality through use case diagrams and descriptions, followed by an initial class diagram that outlines the system's structure and supports implementation planning.

The project also considers potential design limitations before implementation and incorporates version control practices using GitHub to reflect real-world software development workflows. GitHub link: <https://github.com/Thanh526/Thanh-Tran---24247169---Assessment-Phase-1---Project-Progress-Report-and-Presentation>

Overall, this project demonstrates how structured modelling supports the software development lifecycle and contributes to building scalable, maintainable, and reliable systems.

2. Organisational and system analysis

2.1. Overview of the selected organisation

The selected organisation for this project is Auckland University of Technology (AUT), a prominent public university formally established in 2000 in New Zealand. AUT is recognised as one of the nation's largest tertiary institutions, with 26,708 enrolled domestic and international students and over 4,046 staff members in 2025. The university operates across multiple campuses and delivers a wide range of undergraduate and postgraduate programmes, reflecting a high degree of operational scale and complexity (AUT, n.d.-a).



Figure 1: AUT's building (AUT, 2018)

2.2. Departmental structure and responsibilities

AUT has a well-established organisational structure consisting of multiple departments that support its academic and administrative functions, including but not limited to the following:

- **Academic Departments (Faculties):** Responsible for course delivery, curriculum design, research activities, and maintaining academic standards across 5 faculties and 15 schools (AUT, n.d.-b).
- **Student Services Department:** Provides student support including enrolment assistance, academic advice, counselling, and career support (AUT, n.d.-c).
- **Academic Registry Department:** Manages enrolments, course registration, academic records, timetables, and graduation processes (AUT, n.d.-d).
- **IT Department:** Responsible for managing university-wide digital infrastructure and services, including network access, user authentication, email systems, cybersecurity, printing services, and technical support (AUT, n.d.-e).
- **Finance Department:** Responsible for managing institutional budgeting, financial planning, tuition fee administration, procurement support, financial reporting, and allocation of university resources (AUT, n.d.-f).

The presence of these distinct functional areas requires coordinated operations supported by integrated information systems, such as the Course Enrolment System, Course Information System, and Finance System. Among these systems, the Course Enrolment System is selected as the focus of this project.

2.3. Problem statement

The current enrolment process at AUT relies heavily on manual, paper-based procedures for managing student enrolment and course-related processes. This approach creates several operational challenges, including lost or misplaced enrolment forms, manual prerequisite checks prone to errors, delays in enrolment approval—especially during peak periods—and difficulty tracking student progress or course capacity. These inefficiencies result in slow processing times, inconsistent data, limited accessibility, and poor coordination between departments.

To address these issues, AUT requires a Course Enrolment System to manage course selection, enrolment, and related academic processes. The system will

integrate with existing university systems, including a Course Information System and a Finance System, to retrieve course data and payment status. The system must operate across multiple departments, including the Academic Departments, Academic Registry Department, and Finance Department, each responsible for different aspects of the enrolment workflow. The system must support both domestic and international students, who may have different enrolment requirements, fee structures, and approval processes.

Students should be able to search for available courses, enrol in them, and view their enrolled courses through the system. When selecting courses, students will be able to view a timetable of the courses they have chosen, and the system will automatically detect any clashes and display them visually. Students can then adjust their selection based on these clashes to ensure a feasible schedule. The system must validate key business rules, such as checking prerequisites, validating course availability, and preventing timetable conflicts before confirming enrolment. To support these processes, the system retrieves student academic records from the Student Information System to validate prerequisite requirements and determine enrolment eligibility. It also retrieves course information from the Course Information System to support course selection, timetable planning, and enrolment validation.

Academic department staff should be able to view course information retrieved from the Course Information System and access that system to manage course details. They ensure that all course-related data is accurate, up to date within the Course Information System, and accessible to students during the enrolment process. The system retrieves staff role and authorisation information from the HR System to determine whether users are permitted to access academic department functions, ensuring that only authorised staff can view and manage course-related information.

Academic Registry staff are responsible for reviewing and approving student enrolment requests. The system provides enrolment information such as prerequisite status, course availability, and timetable conflict results to support their decision-making. Staff approve or reject enrolments based on academic requirements (such as prerequisites and programme structure) and administrative

rules (such as course capacity, timetable constraints, and enrolment deadlines) to ensure compliance with university policies. The system retrieves student academic records from the Student Information System to support the enrolment validation process, including prerequisite and programme checks.

Finance staff should be able to monitor payment status retrieved from the Finance System and access that system to handle exceptional cases such as failed or delayed payments. For international students, the system automatically confirms enrolment only after successful course fee and insurance verification retrieved from the Finance System. For domestic students, the system can automatically confirm enrolment before payment of course fee based on applicable university financial rules and payment arrangements recorded in the Finance System.

The system must support seamless integration between these components to ensure a consistent data flow, as well as secure and real-time data sharing among all stakeholders.

The proposed system will be a web-based application accessible through a browser on any device. It must support multiple concurrent users while ensuring data accuracy, security, and real-time updates. By replacing the existing manual process, the system aims to improve operational efficiency, reduce errors, and enhance the overall user experience for both domestic and international students, as well as staff.

2.4. Stakeholders and user roles

Based on the problem statement, the key stakeholders and their respective roles are identified as follows:

- **Student (Domestic / International):** Search for available courses, view timetables and detect clashes, enrol in courses, and manage enrolment requests
- **Academic Department staff:** View course information retrieved from the Course Information System and access that system to manage course details, including course descriptions, prerequisites, capacity limits, and timetable schedules

- **Academic Registry staff:** Review and approve enrolment requests in accordance with academic and administrative rules, using enrolment and student academic information retrieved from the Course Information System and Student Information System
- **Finance staff:** Monitor payment status retrieved from the Finance System and access that system to handle exceptional cases
- **IT staff:** Maintain system infrastructure, ensure availability, and provide technical support
- **System Administrators:** Manage user accounts, access control, ensure system uptime and security

This diversity of stakeholders highlights the need for a system that supports multiple user roles with controlled access and responsibilities.

2.5. System scope, boundaries, and assumptions

The Course Enrolment System is designed to address the inefficiencies in the current manual enrolment process, reduce errors in prerequisite verification and course allocation, improve coordination between departments, and enhance the accessibility and overall enrolment experience for both domestic and international students through integration with existing university systems.

i. System scope

The proposed system supports core enrolment operations across multiple departments, including Academic Departments, Academic Registry Department, and Finance Department, and integrates with existing university systems to retrieve student, course, staff, and payment data. Specifically, the system will:

- Support course search, selection, and enrolment
- Validate business rules such as prerequisites, course capacity, and timetable conflict detection
- Support enrolment approval workflows across departments
- Integrate with the Student Information System, Course Information System, HR System, and Finance System for data retrieval

- Provide timetable visualisation and real-time enrolment status updates

ii. System boundaries

The system is limited to student enrolment and related administrative processes. It does not include academic grading, transcript generation, teaching delivery, or staff payroll management.

The system does not manage or modify student, staff, course or payment data directly, as these functions are handled by integrated systems. Instead, it retrieves and displays relevant information from these systems to support enrolment operations.

iii. Assumptions

The following assumptions are made for the system:

- All users have access to a web browser and stable internet connection.
- Users are pre-authenticated by the university before accessing the system.
- All departments follow standardised workflows supported by the system.
- Course and student data provided by integrated systems are accurate and up to date.
- The system supports concurrent access by multiple users without data inconsistency.
- The system integrates with AUT's existing systems, including Student Information System, Course Information System, HR System, and Finance System, to retrieve student, course, staff and payment data.

3. Requirements engineering

3.1. Requirements elicitation

The requirements for the Course Enrolment System were elicited using a combination of problem statement analysis, existing document analysis, observation of existing system, and questionnaires for stakeholder to ensure accuracy, completeness, and traceability.

i. Problem statement analysis

Firstly, the problem statement was analysed to identify the key system requirements based on the requirements from AUT, several system requirements were identified:

- The need for students to search, select, and enrol in courses indicates that the system shall provide functionality for course search, course selection, and enrolment management.
- The requirement for students to view their enrolled courses and timetable suggests that the system shall provide timetable visualisation and allow students to view and manage their enrolled courses.
- The need to check prerequisites, course availability, and timetable conflicts indicates that the system shall retrieve student academic records from the Student Information System and retrieve course information from the Course Information System to automatically validate prerequisite, check course capacity, and detect timetable clashes before confirming enrolment.
- The requirement for academic department staff to manage course information indicates that the system shall allow staff to view course information retrieved from the Course Information System and provide access to that system to manage course details. The system shall also retrieve staff role and authorisation information from the HR System to enforce role-based access control.

- The need for academic registry staff to review and approve enrolments indicates that the system shall support an enrolment approval workflow based on academic and administrative rules.
- The requirement for finance staff to monitor and manage payments indicates that the system shall retrieve payment status from the Finance System, while allowing finance staff to access the Finance System to handle exceptional cases such as failed or delayed payments.
- The need for automatic enrolment confirmation for both international and domestic students indicates that the system shall retrieve student information from the Student Information System and financial information from the Finance System to determine enrolment eligibility and confirm enrolment according to applicable payment rules for different student types.
- The need for coordination across multiple departments indicates that the system shall support integrated workflows and real-time data sharing between departments.
- The requirement to support both domestic and international students suggests that the system shall handle different enrolment rules, fee structures, and approval processes.
- The requirement for a web-based system accessible on multiple devices indicates that the system shall be accessible through a web browser and support multiple concurrent users.
- The need for improved efficiency, accuracy, and user experience indicates that the system shall provide real-time updates, ensure data consistency, and reduce manual processing.

This analysis ensures that the identified requirements directly address the key problems in the current system and support the development of a more efficient and reliable enrolment process.

ii. Existing document analysis

Secondly, document analysis was conducted using the “Enrolling in courses and classes” guideline published by AUT. The analysed document outlines the student

enrolment process, course selection rules, and administrative constraints that govern how students enrol in courses and classes (AUT, n.d.-d).

From this document, several explicit system requirements were identified and derived based on the recommendations provided:

- Students can search for courses by name or topic leads to the requirement that the system shall provide course search functionality based on multiple criteria (e.g., course name, topic, or programme).
- Students should not enrol in courses with timetable clashes leads to the requirement that the system shall automatically detect and notify students of timetable conflicts during course selection.
- Students must meet prerequisite requirements before enrolling leads to the requirement that the system shall validate prerequisite completion before allowing enrolment.
- Students should enrol in compulsory courses first due to limited capacity leads to the requirement that the system shall display course capacity limits and real-time availability of course slots.
- Students should plan enrolment across the academic year leads to the requirement that the system shall provide timetable visualisation and course scheduling information to support enrolment planning.

This document analysis ensures that the proposed system aligns with actual institutional policies and student guidance instead of assumptions. It also provides a reliable foundation for defining business rules that the system must follow.

iii. Observation of existing system

Thirdly, observation of the current manual workflow for the course enrolment process at AUT has been conducted. The current process relies on a combination of paper forms, email communication, and spreadsheet tracking between different departments, which results in delays, inconsistencies, and limited ability to track enrolment status in real time.

In the current process, students first obtain enrolment forms online or from faculty offices. They manually fill in the course selections and submit the forms to the Academic Registry Department for review. Academic registry staff then manually check student eligibility by referring to the Student Information System, Course Information System, and spreadsheets to confirm prerequisites, programme requirements, and course availability. If any issues are identified, such as missing prerequisites or timetable clashes, the form is returned to the student for correction, causing additional delays.

Course availability is managed by academic department staff. The process is done manually, so course availability information is not always updated in real time. This may lead to students attempting to enrol in courses that are already full. Checking for timetable clashes is also done manually by comparing course schedules, which increases the risk of human error.

After enrolment is approved, the enrolment request is forwarded to the Finance Department. Students are required to make payments through the Finance System, and payment status is manually verified by finance staff before enrolment is confirmed. This process can be delayed due to missing payment records or differences in processing time between systems.

From this observation, several system requirements were identified:

- The rely on spreadsheets and separate systems for course availability indicates that the system shall maintain real-time course capacity updates.
- The manual checking of prerequisites and programme requirements indicates that the system shall retrieve student academic records from the Student Information System to automatically validate student eligibility before enrolment submission.
- The manual comparison of timetables indicates that the system shall automatically detect and prevent timetable clashes during course selection.
- The need to submit and resubmit forms indicates that the system shall allow students to modify and resubmit enrolment requests within the system.

- The separate handling of payments and manual verification indicates that the system shall integrate with the Finance System and automatically retrieve payment status.
- The manual verification of payments and delays in enrolment confirmation indicates that the system shall automatically confirm enrolment only after successful payment verification.
- The limited ability to track enrolment status in real time indicates that the system shall provide real-time tracking of enrolment status for students and staff.
- The reply on the other systems for eligibility checking indicates that the system shall integrate with those systems to retrieve student, course, and payment data.
- The lack of a unified system across departments indicates that the system shall support integrated workflows and real-time data sharing between Academic Departments, Academic Registry Department, and Finance Department.
- The involvement of multiple departments in the enrolment process indicates that the system shall support role-based access for different user types.

This observation highlights inefficiencies in the current manual process and provides a clear foundation for designing a more efficient, automated, and integrated enrolment system.

iv. Stakeholder interview

Besides the above requirements elicitation methods, stakeholder interviews (simulated) were considered to gather further insights into system administration, security, and technical requirements that are not clearly visible in the current enrolment workflow.

The interview was conducted with academic registry staff, IT support staff, and system administration personnel involved in the operation of the proposed system.

The following questions and answers were conducted during the interview:

- **Academic department staff interview:**

- **Question:** How do you handle situations where students are unable to enrol for courses due to issues such as not meeting prerequisites or timetable clashes?

Answer: In those cases, students may consult with me. Based on academic judgement and programme requirements, I may grant an override (e.g., prerequisite override or timetable exception). Once the override is granted, the student is allowed to proceed with the enrolment process and enrol in the course themselves in special circumstances, such as prior learning, equivalent experience, or approved timetable exceptions.

- **Academic registry staff interview:**

- **Question:** After an override is granted by academic department staff, what actions do you take in processing the enrolment request?

Answer: After an override is granted, I review the enrolment request to ensure compliance with all academic and administrative rules, including programme requirements, programme structure, course capacity, the student's enrolment load for the semester, and timetable constraints. Based on this evaluation, I approve or reject the enrolment request in accordance with university policies.

- **Question:** Are there any different requirements you consider for international and domestic students beyond the standard academic rules?

Answer: For domestic students, I primarily focus on standard academic requirements and institutional enrolment policies, including course progression rules and a maximum study load of 75 credits per semester. For international students, I also consider additional requirements such as visa conditions and study load limits, where full-time students are generally allowed up to 60

credits per semester, while part-time study (e.g., exchange or approved reduced load) may be limited to 30 credits per semester.

- **System administrator interview:**

- **Question:** What level of control is required for managing user accounts and access permissions across different roles?

Answer: User accounts must be managed using role-based access control. Each user type, such as students, academic registry staff, academic department staff, finance staff, IT staff, and system administrators, is assigned specific permissions based on their responsibilities. This ensures users can only access functions relevant to their role.

- **Question:** What security measures are required to ensure system activity can be tracked and audited?

Answer: The system should maintain audit logs that record all important user activities such as enrolment actions, approvals, course updates, and system configuration changes. These logs support security monitoring, accountability, and troubleshooting when required.

- **Question:** How should the system support configuration of academic rules such as enrolment periods and course limits?

Answer: The system should allow system administrators to configure academic rules such as enrolment periods, course capacity limits, prerequisite rules, and enrolment deadlines. These settings ensure the system can be adjusted for different academic terms without requiring major system changes.

- **IT staff interview:**

- **Question:** What technical responsibilities are required from you to maintain system availability and integration with external systems?

Answer: I am responsible for maintaining system uptime, performance, and database integrity. I also manage integration with the university's central database and external systems to ensure

accurate and consistent data exchange across enrolment, course management, and payment services.

From the responses, several system requirements were identified:

- The need to handle exceptional academic cases indicates that the system shall allow academic department staff to grant prerequisite overrides or timetable exceptions, enabling students to proceed with enrolment requests.
- The requirement for secure access to system functions indicates that the system shall support role-based access control for different user types, including students, academic staff, academic registry staff, finance staff, IT staff, and system administrators.
- The requirement for secure access to system functions indicates that the system shall provide a login mechanism for all user types.
- The need for security monitoring and accountability indicates that the system shall maintain audit logs of user activities for tracking system usage and supporting security compliance.
- The requirement for system stability and operational support indicates that IT staff shall be able to monitor system performance, maintain system uptime, and resolve technical issues.
- The need for flexible system configuration indicates that system administrators shall be able to configure system-wide settings such as enrolment periods, course capacity limits, and enrolment rules.
- The requirement for integration management indicates that IT staff shall manage integration between the Course Enrolment System and other systems, such as Finance System and Course Information System, to ensure accurate data exchange.

3.2. User and system requirements

i. User requirements

Based on the above requirements elicitation methods, the user requirements that describe what each user needs to do with the system are identified as follows:

- Students should be able to:
 - Log in to the system using valid credentials
 - Search for available courses based on different criteria such as course name, topic, or programme.
 - View course details, including prerequisites, timetable, and course availability.
 - Select and enrol in courses.
 - View their enrolled courses and timetable.
 - Receive notifications or feedback when enrolment issues occur, such as timetable clashes, missing prerequisites, or full courses.
 - Modify and resubmit enrolment requests if changes are required.
 - Track the status of their enrolment in real time.
- Academic department staff should be able to:
 - Log in to the system using valid credentials
 - View course information retrieved from the Course Information System.
 - Access the Course Information System to manage course details.
 - Grant prerequisite overrides or timetable exceptions to allow students to proceed with enrolment requests.
- Academic registry staff should be able to:
 - Log in to the system using valid credentials
 - Review and process student enrolment requests, including those with academic department overrides (e.g., prerequisite or timetable exceptions).
 - Check student eligibility using data retrieved from the Student Information System, including prerequisites, programme requirements, and enrolment rules.
 - Approve or reject enrolment requests based on academic and administrative rules, ensuring compliance with university policies.
 - Monitor enrolment status and ensure the enrolment process follows the required workflow.

- Finance staff should be able to:
 - Log in to the system using valid credentials
 - Monitor student payment status retrieved from the Finance System.
 - Handle exceptional cases such as missing, failed, or delayed payments via the Finance System.
- System administrators should be able to:
 - Log in to the system using valid credentials
 - Manage User Roles and Access Control using role and identity information retrieved from the HR System and Student Information System.
 - Configure system-wide settings such as enrolment periods, course limits, and system rules.
 - Review system audit logs for security compliance.
- IT staff should be able to:
 - Log in to the system using valid credentials
 - Maintain system uptime and database integrity.
 - Manage integration with the Student Information System, Course Information System, HR System, and Finance System to support accurate and consistent data exchange.
 - Monitor system performance and resolve technical issues.

ii. System requirements

Based on the above requirements elicitation methods, the system requirements that describe what the system shall do are identified as follows:

- The system shall integrate with the Student Information System and the Course Information System to retrieve student academic records and course information to automatically validate prerequisites, course capacity, and timetable conflicts, and provide notifications to students before enrolment confirmation.
- The system shall support an enrolment override workflow where academic department staff can submit prerequisite or timetable exceptions, and

academic registry staff can review and approve or reject enrolment requests based on these overrides and academic policies.

- The system shall integrate with the Finance System to retrieve payment status and automatically confirm enrolment according to applicable financial rules, including successful payment verification for international students and policy-based payment arrangements for domestic students.
- The system shall integrate with the Student Information System and the HR System to enforce role-based access control by managing user roles and permissions for all user types, ensuring that users can only access system functions and data relevant to their assigned roles.

3.3. Functional requirements

Based on the user and system requirements, the functional requirements table of the Course Enrolment System are defined as follows:

ID	Functional Requirement	Priority
FR1	The system shall allow users to securely log in using valid credentials	High
FR2	The system shall allow students to search for available courses based on criteria such as course name, topic, or programme	High
FR3	The system shall retrieve course information from the Course Information System	High
FR4	The system shall allow students to view retrieved course information, including prerequisites, timetable schedules, and course availability	High
FR5	The system shall allow students to select courses	High
FR6	The system shall allow students to enrol in courses	High
FR7	The system shall allow students to view list of enrolled courses	High
FR8	The system shall provide timetable visualisation of enrolled courses for students during enrolment	High
FR9	The system shall retrieve student information from the Student Information System	High
FR10	The system shall automatically validate prerequisite requirements during enrolment using student information	High
FR11	The system shall automatically check course capacity during enrolment	High
FR12	The system shall automatically detect timetable clashes during enrolment	High
FR13	The system shall notify students of enrolment issues such as timetable conflicts, missing prerequisites, or courses being full	High

FR14	The system shall allow students to modify and resubmit enrolment requests	Medium
FR15	The system shall provide real-time tracking of enrolment status	High
FR16	The system shall allow academic department staff to view course information	Medium
FR17	The system shall allow academic department staff to access the Course Information System via the Course Enrolment System to manage course details	Medium
FR18	The system shall allow academic department staff to grant prerequisite overrides or timetable exceptions	High
FR19	The system shall allow academic registry staff to monitor enrolment status across all student applications	Medium
FR20	The system shall allow academic registry staff to review individual enrolment requests	High
FR21	The system shall allow academic registry staff to check student eligibility using student information, including prerequisites, programme requirements, and enrolment rules	High
FR22	The system shall allow academic registry staff to approve or reject enrolment requests	High
FR23	The system shall retrieve student financial information, such as course and insurance payment status, and payment due information, from the Finance System	High
FR24	The system shall display payment status within the enrolment status view	Medium
FR25	The system shall allow finance staff to access the Finance System via the Course Enrolment System to manage exceptional cases, such as failed or delayed payments	Medium
FR26	The system shall confirm enrolment for international students only after successful payment verification	High
FR27	The system shall confirm enrolment for domestic students based on applicable financial policies	High
FR28	The system shall retrieve HR information from the HR System	High
FR29	The system shall provide role-based access control for different user roles base on HR and student information	High
FR30	The system shall allow system administrators to configure system-wide settings such as enrolment periods and course limits	Medium
FR31	The system shall maintain audit logs of system activities	Medium
FR32	The system shall support IT staff in maintaining system uptime to ensure continuous system availability	High
FR33	The system shall support IT staff in ensuring database integrity, including consistency and accuracy of stored data	High
FR34	The system shall support IT staff in monitoring system performance	High
FR35	The system shall support IT staff in resolving technical issues	High

Table 1: Functional requirements table

3.4. Non-functional requirements

To ensure that the system is secure, reliable, efficient, and usable across multiple user groups and integrated systems, the non-functional requirements table of the Course Enrolment System are defined in accordance with ISO/IEC 25010:2023 (ISO, 2023):

ID	Non-Functional Requirement	ISO/IEC 25010:2023 Attribute	Priority
NFR1	The system shall support at least 5,000 simultaneous users during peak enrolment periods	Performance efficiency (capacity)	High
NFR2	The system shall respond to course search requests within 2 seconds under normal load	Performance efficiency (time behaviour)	High
NFR3	The system shall complete enrolment validation (prerequisite check, capacity check, timetable clash detection) within 3 seconds	Performance efficiency (time behaviour)	High
NFR4	The system shall ensure all user authentication processes (login and identity verification) complete within 2 seconds	Performance efficiency (time behaviour)	High
NFR5	The system shall ensure data synchronisation across integrated systems within 5 seconds to maintain data consistency	Performance efficiency (time behaviour)	High
NFR6	The system shall provide secure authentication using hashed passwords for login and secure tokens for session management	Security (confidentiality)	High
NFR7	The system shall ensure that all sensitive data (student, academic, and financial) is transmitted securely using HTTPS	Security (integrity)	High
NFR8	The system shall ensure that all sensitive data is protected when temporarily stored within the system	Security (integrity)	High
NFR9	The system shall ensure audit logs are retained for at least 7 years for compliance purposes	Security (accountability)	Medium
NFR10	The system shall automatically log out inactive users after 15 minutes of inactivity	Security (confidentiality)	Medium
NFR11	The system shall be available 24/7 with 99.5% uptime, excluding scheduled maintenance	Reliability (availability)	High

NFR12	The system shall ensure scheduled maintenance downtime does not exceed 4 hours per month	Reliability (availability)	Medium
NFR13	The system shall perform daily automated backups of all enrolment and user data	Reliability (recoverability)	High
NFR14	The system shall support data recovery within 1 hour in case of system failure	Reliability (recoverability)	High
NFR15	The system shall support access from standard web browsers (Chrome, Edge, Firefox) without additional plugins	Compatibility (interoperability)	Medium
NFR16	The system shall support integration with other AUT systems, including Student Information System, Course Information System, HR System, and Finance System	Compatibility (interoperability)	High
NFR17	The system shall support scalable architecture to allow future increase to 10,000+ concurrent users without redesign	Flexibility (scalability)	Medium
NFR18	The system shall provide a user-friendly interface that allows users to complete enrolment tasks with minimal training	Interaction capability (learnability)	Medium
NFR19	The system shall provide clear error messages and guidance when enrolment issues occur	Interaction capability (operability)	Medium

Table 2: Non-functional requirements tab

3.5. Traceability between stakeholders and requirements

Based on the functional and non-functional requirements defined in Sections 3.3 and 3.4, a traceability table is constructed below to illustrate which stakeholders are responsible for or affected by each functional and non-functional requirement.

Stakeholder	Functional Requirements	Non-Functional Requirements
Students	FR1 - FR15, FR24, FR26, FR27	NFR1 - NFR8, NFR10, NFR11 - NFR19
Academic department staff	FR1, FR3, FR16 - FR18	NFR4, NFR6 - NFR8, NFR15, NFR16, NFR18, NFR19
Academic registry staff	FR1, FR9, FR19 - FR22	NFR6 - NFR8, NFR11, NFR15, NFR16, NFR18, NFR19
Finance staff	FR23 - FR27	NFR6 - NFR8, NFR11, NFR15, NFR16, NFR18, NFR19
IT staff	FR32 - FR35	NFR11 - NFR19
System Administrators	FR28 – FR31	NFR6 - NFR14, NFR16 - NFR19

Table 3: Traceability between stakeholders and requirements

4. UML requirements modelling

After completing the requirements analysis in part 3, the next step in the system development process is UML requirements modelling. This stage focuses on representing the system functionality from a user perspective using use case modelling. It translates the identified functional requirements into a structured visual representation that illustrates how different actors interact with the system.

The use case model provides a clear understanding of system behaviour, defines system boundaries, and ensures that all stakeholder requirements are properly captured. It also serves as a foundation for subsequent system design and implementation.

4.1. Actor

Based on the functional requirements defined, the following actors are identified for the Course Enrolment System:

- **Primary actors:**
 - **Student:** Responsible for logging into the system (FR1), searching courses (FR2), viewing course details (FR4), selecting courses (FR5), submitting enrolment requests (FR6), viewing list of enrolled courses (FR7), viewing timetable (FR8), receiving enrolment issue notifications (FR13), modifying enrolment requests (FR14), and tracking enrolment status (FR15)
 - **Academic department staff:** Responsible for logging into the system (FR1), viewing course information (FR16), managing courses via the Course Information System (FR17), and granting prerequisite overrides or timetable exceptions (FR18)
 - **Academic registry staff:** Responsible for logging into the system (FR1), monitoring enrolment status (FR19), reviewing enrolment requests (FR20), checking student eligibility (FR21), and approving or rejecting enrolment requests (FR22)
 - **Finance staff:** Responsible for logging into the system (FR1), viewing payment information (FR24) and handling exceptional cases via the Finance System (FR25)
 - **System administrator:** Responsible for logging into the system (FR1), managing role-based access control (FR29), configuring system settings (FR30), and reviewing audit logs (FR31)

- **IT Staff:** Responsible for logging into the system (FR1), maintaining system uptime (FR32), ensuring database integrity (FR33), monitoring system performance (FR34), and resolving technical issues (FR35)
- **External system actors:**
 - **Student Information System:** Provides student information (FR8)
 - **Course Information System:** Provides course information (FR2)
 - **Finance System:** Provides financial data (FR22)
 - **HR System:** Provides staff role and authorisation data (FR27)

4.2. Use case

i. Actor – use case

Based on the defined functional requirements and actors, the following actor – use case mapping table are identified for the Course Enrolment System. Each use case represents a specific system function that supports user interaction and system behaviour:

Actor	Use Case ID	Use Case
Student	UC1	Login
	UC2	Search Courses
	UC3	View Course
	UC4	Select Courses
	UC5	Submit Enrolment
	UC6	View Selected List
	UC7	View Timetable
	UC8	Modify Enrolment Request
	UC9	Track Enrolment Status
Academic department staff	UC1	Login
	UC3	View Course
	UC10	Manage Courses
	UC11	Grant Override
Academic registry staff	UC1	Login
	UC12	Monitor Enrolment Status
	UC13	Review Enrolment Requests
	UC14	Check Student Eligibility
	UC15	Approve Enrolment Request
Finance staff	UC1	Login
	UC16	View Payment Information
	UC17	Handle Payment Exceptions
System administrator	UC1	Login
	UC18	Manage Access Control
	UC19	Configure System Settings
	UC20	Review Audit Logs

IT staff	UC1	Login
	UC21	Maintain System Uptime
	UC22	Ensure Database Integrity
	UC23	Monitor System Performance
	UC24	Resolve Technical Issues

Table 4: Actor - use case mapping

ii. System – use case

Besides the use cases in the actor–use case mapping table, there are additional use cases implemented by the system, as shown in the system – use case table below

Use Case ID	Use Case
UC25	Retrieve Student Information
UC26	Retrieve Course Information
UC27	Retrieve Financial Information
UC28	Retrieve HR Information
UC29	Notify Enrolment Issues
UC30	Validate Course Prerequisites
UC31	Check Course Capacity
UC32	Detect Timetable Clashes
UC33	Confirm Enrolment
UC34	Maintain Audit Logs

Table 5: System – use case

iii. Use case descriptions

The following tables provide detailed descriptions of some of the use cases identified above, focusing on the core system functions that are critical to the Course Enrolment System.

ID	UC1
Name	Login
Description	User securely accesses the system using valid credentials. The system authenticates the user and assigns appropriate role-based access.
Pre-condition	System is available User has a valid system account
Event flow	1. User opens the login page 2. User enters username and password 3. User clicks on login button 4. System validates credentials 5. System retrieves user role from Student Information System and HR System (UC25, UC28) 6. System grants access to the system dashboard based on role

Includes	UC25 “Retrieve student information” UC28 “Retrieve HR Information” UC34 “Maintain audit logs”
Extension points	Invalid credentials → system displays error message Account locked → system denies access
Triggers	
Post-condition	User is successfully authenticated and logged into the system

Table 6: Use case “Login” descriptions

ID	UC2
Name	Search Courses
Description	Student searches for available courses based on criteria such as course name, topic, or programme
Pre-condition	System is available Student is logged in (UC1)
Event flow	1. Student enters search criteria 2. Student clicks on search button 3. System retrieves course information from Course Information System (UC26) 4. System displays matching course results
Includes	UC26 “Retrieve course information”
Extension points	No matching results → system displays a no results message
Triggers	
Post-condition	A list of matching courses is displayed to the student

Table 7: Use case “Search Course” descriptions

ID	UC3
Name	View Course
Description	Student view detailed information of courses that they intend to enrol in
Pre-condition	Student has searched for courses Course search results are available
Event flow	1. Student clicks on a course from the search results 2. System retrieves the course information from Course Information System (UC26) 3. System displays course details to the student
Includes	UC26 “Retrieve course information”
Extension points	Course information unavailable → system displays error message
Triggers	Student clicks a course to view
Post-condition	Course details are displayed to the student

Table 8: Use case “View Course” descriptions

ID	UC4
Name	Select Courses

Description	Student selects available courses that they intend to enrol in
Pre-condition	Student has searched for courses Course search results are available
Event flow	1. Student selects one or more courses in search list 2. System retrieves student and courses information (UC25, UC26) 3. System validates prerequisite requirements (UC30) 4. System checks course capacity (UC31) 5. System detects timetable clashes (UC32) 6. System displays selected courses timetable to the student (UC7) 7. System temporarily stores validation selected courses in the enrolment cart
Includes	UC25 "Retrieve student information" UC26 "Retrieve course information" UC30 "Validate course prerequisites" UC31 "Check course capacity" UC32 "Detect timetable clashes" UC7 "View timetable"
Extension points	UC8 "Receive enrolment issue notifications"
Triggers	Student adds a course to the enrolment list
Post-condition	Selected courses are stored in a draft enrolment list

Table 9: Use case "Select Course" descriptions

ID	UC5
Name	Submit Enrolment
Description	Student submits selected courses for enrolment processing
Pre-condition	Student has selected courses
Event flow	1. Student clicks submit enrolment request button 2. System checks if there are existing unresolved enrolment issue notifications 3. If issues exist, system retrieves latest validation status 4. System performs final validation check if required (capacity, clashes, prerequisites) 5. If no enrolment issue exist, system creates enrolment request
Includes	UC25 "Retrieve student information" UC26 "Retrieve course information" UC30 "Validate course prerequisites" UC31 "Check course capacity" UC32 "Detect timetable clashes" UC34 "Maintain audit logs"
Extension points	UC8 "Receive enrolment issue notifications"
Triggers	Student clicks submit enrolment request button
Post-condition	Enrolment request is created with status "Pending"

Table 10: Use case "Submit Enrolment" descriptions

4.3. Use case relationships

The Course Enrolment System use case model uses include, extend, and generalisation relationships to represent how different system functions interact with each other.

i. Include relationships

The include relationship is used when one use case must include another use case in its normal flow.

The following include relationships are identified in the system:

- “Login” must include “Retrieve Student Information”
- “Login” must include “Retrieve HR Information”
- “Login” must include “Maintain Audit Logs”
- “Search Courses” must include “Retrieve Course Information”
- “View Course” must include “Retrieve Course Information”
- “Select Courses” must include “Retrieve Student Information”
- “Select Courses” must include “Retrieve Course Information”
- “Select Courses” must include “Validate Course Prerequisites”
- “Select Courses” must include “Check Course Capacity”
- “Select Courses” must include “Detect Timetable Clashes”
- “Select Courses” must include “View Timetable”
- “Submit Enrolment” must include “Retrieve Student Information”
- “Submit Enrolment” must include “Retrieve Course Information”
- “Submit Enrolment” must include “Validate Course Prerequisites”
- “Submit Enrolment” must include “Check Course Capacity”
- “Submit Enrolment” must include “Detect Timetable Clashes”
- “Submit Enrolment” must include “Maintain Audit Logs”
- “View Timetable” must include “Retrieve Course Information”
- “Modify Enrolment Request” must include “Retrieve Student Information”
- “Modify Enrolment Request” must include “Retrieve Course Information”
- “Modify Enrolment Request” must include “Validate Course Prerequisites”
- “Modify Enrolment Request” must include “Check Course Capacity”
- “Modify Enrolment Request” must include “Detect Timetable Clashes”
- “Modify Enrolment Request” must include “Maintain Audit Logs”
- “Track Enrolment Status” must include “Retrieve Student Information”
- “Manage Courses” must include “View Course”
- “Monitor Enrolment Status” must include “Retrieve Student Information”
- “Review Enrolment Requests” must include “Retrieve Student Information”

- “Check Student Eligibility” must include “Retrieve Student Information”
- “View Payment Information” must include “Retrieve Financial Information”
- “Confirm Enrolment” must include “Retrieve Student Information”
- “Confirm Enrolment” must include “Retrieve Course Information”
- “Confirm Enrolment” must include “Retrieve Financial Information”
- “Confirm Enrolment” must include “Validate Course Prerequisites”
- “Confirm Enrolment” must include “Check Course Capacity”
- “Confirm Enrolment” must include “Detect Timetable Clashes”
- “Confirm Enrolment” must include “Approve Enrolment Request”
- “Manage Access Control” must include “Retrieve HR Information”
- “Manage Access Control” must include “Retrieve Student Information”
- “Review Audit Logs” must include “Maintain Audit Logs”

ii. Extend relationships

The extend relationship is used when a use case may be triggered under certain conditions such as validation failure, exceptions, or additional processing steps.

The following extend relationships are defined:

- “Select Courses” may extend to “Notify Enrolment Issues”
- “Submit Enrolment” may extend to “Notify Enrolment Issues”
- “Submit Enrolment” may extend to “Modify Enrolment Request”
- “Modify Enrolment Request” may extend to “Notify Enrolment Issues”
- “View Payment Information” may extend to “Handle Payment Exceptions”
- “Handle Payment Exceptions” may extend to “Confirm Enrolment”

iii. Generalisation relationships

The generalisation relationship is used when actors or use cases share common characteristics and can be represented under a single generalised entity to reduce redundancy and improve model clarity.

The following generalisation relationships are defined:

- “Staff” is a generalised actor with specialised actors “Academic Department Staff”, “Academic Registry Staff”, “Finance Staff”, “IT Staff”, and “System Administrator”
- “External System” is a generalised actor with specialised actors “Student Information System”, “Course Information System”, “Finance System”, and “HR System”

4.4. Use Case Diagram

After defining the actors, use cases, and their relationships, the use case diagram of the Course Enrolment System is constructed and presented in the figure below.

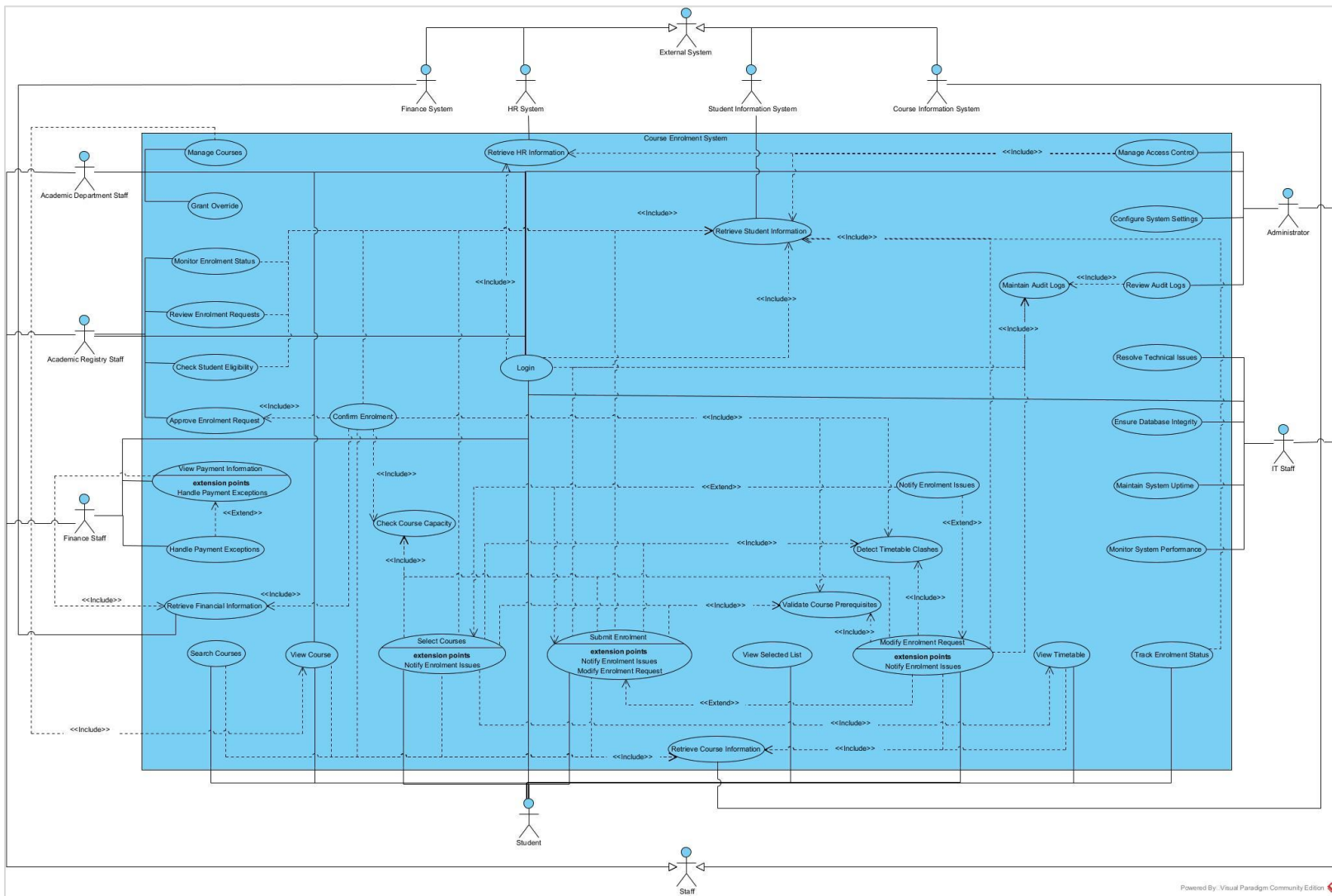


Figure 2: Course Enrolment System's Use Case Diagram

The use case diagram above provides an overview of the Student Enrolment System, showing how different actors interact with the system within the defined system boundary. It illustrates key system functions and the relationships between use cases, ensuring that all functional requirements are clearly represented.

5. Initial structural design

5.1. Class diagram

After completing the requirements analysis and use case modelling, the next step is to develop the class diagram of the Course Enrolment System.

i. Identify classes

Based on the requirements analysis and use case modelling, the key classes of the Course Enrolment System are identified:

- Core domain classes:
 - User (Abstract)
 - Student
 - Staff (Abstract)
 - AcademicStaff
 - RegistryStaff
 - FinanceStaff
 - ITStaff
 - SystemAdmin
 - Course
 - Timetable
 - EnrolmentRequest
- Service layer classes:
 - EnrolmentService
 - ValidationService
 - TimetableService
 - FinanceService
 - NotificationService
 - AuditService
- Integrated systems:
 - ICourseInfoSystem
 - IStudentInfoSystem
 - IFinanceSystem

- IHRSystem
- Enumerations:
 - StudentType
 - EnrolmentStatus

ii. Identify attributes and operations

Based on the requirements analysis and use case modelling, the attributes and operations of the identified classes are presented in the table below:

Class	Attributes	Operations
User	+userId: String {readOnly} +Username: String -password: String	+login() +logout() +resetPassword()
Student	+studentId: String {readOnly} +programme: String +studentType: StudentType {readOnly} -selectedCourses: Course -enrolmentRequests: EnrolmentRequest	+searchCourses() +selectCourses() +viewCourses() +viewTimetable() +submitEnrolment() +modifyEnrolmentRequest() +viewEnrolmentStatus()
Staff	+staffId: String {readOnly} +department: String	
AcademicStaff	+faculty: String	+viewCourse() +manageCourse() +grantOverride()
RegistryStaff	+officeLocation: String	+monitorEnrolments() +reviewEnrolmentRequest() +checkStudentEligibility() +approveEnrolment() +rejectEnrolment()
FinanceStaff	+financeUnit: String	+viewPaymentStatus() +handlePaymentException()
ITStaff	+technicalArea: String	+monitorSystemPerformance() +maintainSystemUptime() +validateDataConsistency() +resolveTechnicalIssues()
SystemAdmin	+adminLevel: String	+manageUserAccess() +setSystemConfiguration()

		+reviewAuditLogs()
Course	+courseId: String {readOnly} +courseCode: String {readOnly} +courseName: String {readOnly} +description: String +prerequisites: String {readOnly} +capacity: Int {readOnly} +credits: Int {readOnly} +enrolledCount: Int {readOnly}	+isAvailable() +updateCapacity() +getPrerequisites() +incrementEnrolment() { } +addPrerequisite() { } +getFullDetails() { }
Timetable	+timetableId: String {readOnly} +day: String +startTime: String +endTime: String +location: String	+displayTimetable()
EnrolmentRequest	+requestId: String {readOnly} +student_id: String -selectedCourses: Course +submissionDate: Date +enrolmentStatus: EnrolmentStatus	+submit() +resubmit() +cancel() +getRequestedCourses() { } +addCourse() +setStatus()
EnrolmentService		+confirmEnrolment()
ValidationService		+validatePrerequisites() +checkCourseCapacity()
TimetableService		+detectTimetableClashes()
FinanceService		+checkFinancialEligibility(studentId, studentType)
NotificationService		+notifyEnrolmentIssues(student, issues)
AuditService		+logActivity(userId, action, timestamp)
ICourseInfoSystem		+getCourseInfo(courseId) +updateCourse(courseId)
IStudentInfoSystem		+getStudentInfo(studentId) +getAcademicRecord(studentId)
IFinanceSystem		+getPaymentStatus(studentId)

		+verifyInsurance(studentId) +checkFinancialPolicy(studentId)
IHRSystem		+getStaffRole(staffId) +getAccessRights(userId)
StudentType	Domestic International	
EnrolmentStatus	Draft Pending Approved Rejected Confirmed	

Table 11: Classes' attributes and operations

iii. Identify relationships

Based on the requirements analysis and use case modelling, the relationships between classes in the Course Enrolment System are identified and categorised into generalisation, association, dependency, and multiplicity relationships:

- Generalisation Relationships:
 - Student generalises to User
 - Staff generalises to User
 - AcademicStaff generalises to Staff
 - RegistryStaff generalises to Staff
 - FinanceStaff generalises to Staff
 - ITStaff generalises to Staff
 - SystemAdmin generalises to Staff
- Association Relationships
 - Student is associated with Timetable (1 – 1).
 - Each student has only one timetable.
 - Each timetable belongs to one student.
 - Timetable is associated with Course (1 – 1).
 - Each timetable belongs to one course.
 - Each course has only one timetable.
 - Student is associated with EnrolmentRequest (1 – 0..*).
 - Each student can submit zero or many enrolment requests.

- Each enrolment request belongs to exactly one student.
- EnrolmentRequest is associated with Course (1 – 1..*).
- Each enrolment request must contain at least.
 - Each course in the request belongs to one enrolment request.
- EnrolmentRequest is associated with RegistryStaff (0..* – 1).
- Each enrolment request is processed by one registry staff.
 - Each registry staff can handle zero or many enrolment requests.
- Course is associated with AcademicStaff (1..* – 1).
- Each course is managed by one academic staff.
 - Each academic staff can manage multiple courses.
- SystemAdmin is associated with AuditService (1 – 1).
- Each system administrator interacts with the audit service.
 - Each audit service is assigned to only one system administrator.
- Dependency Relationships
 - Student depends on StudentType
 - Student depends on ICourseInfoSystem
 - Course depends on ICourseInfoSystem
 - AcademicStaff depends on ICourseInfoSystem
 - ITStaff depends on ICourseInfoSystem
 - NotificationService depends on Student
 - EnrolmentRequest depends on EnrolmentStatus
 - AcademicStaff depends on EnrolmentRequest
 - EnrolmentService depends on EnrolmentRequest
 - EnrolmentService depends on ValidationService
 - EnrolmentService depends on TimetableService
 - EnrolmentService depends on FinanceService
 - EnrolmentService depends on NotificationService
 - EnrolmentService depends on ICourseInfoSystem
 - EnrolmentService depends on IStudentInfoSystem
 - EnrolmentService depends on IHRSystem
 - EnrolmentService depends on AuditService

- FinanceService depends on IFinanceSystem
- RegistryStaff depends on EnrolmentService
- RegistryStaff depends on IStudentInfoSystem
- ITStaff depends on IStudentInfoSystem
- ITStaff depends on IFinanceSystem
- ITStaff depends on AuditService
- User depends on AuditService
- Staff depends on IHRSystem
- FinanceStaff depends on IFinanceSystem

After defining the classes, attributes, operations, and relationships, the class diagram of the Course Enrolment System is constructed and presented in the figure below.

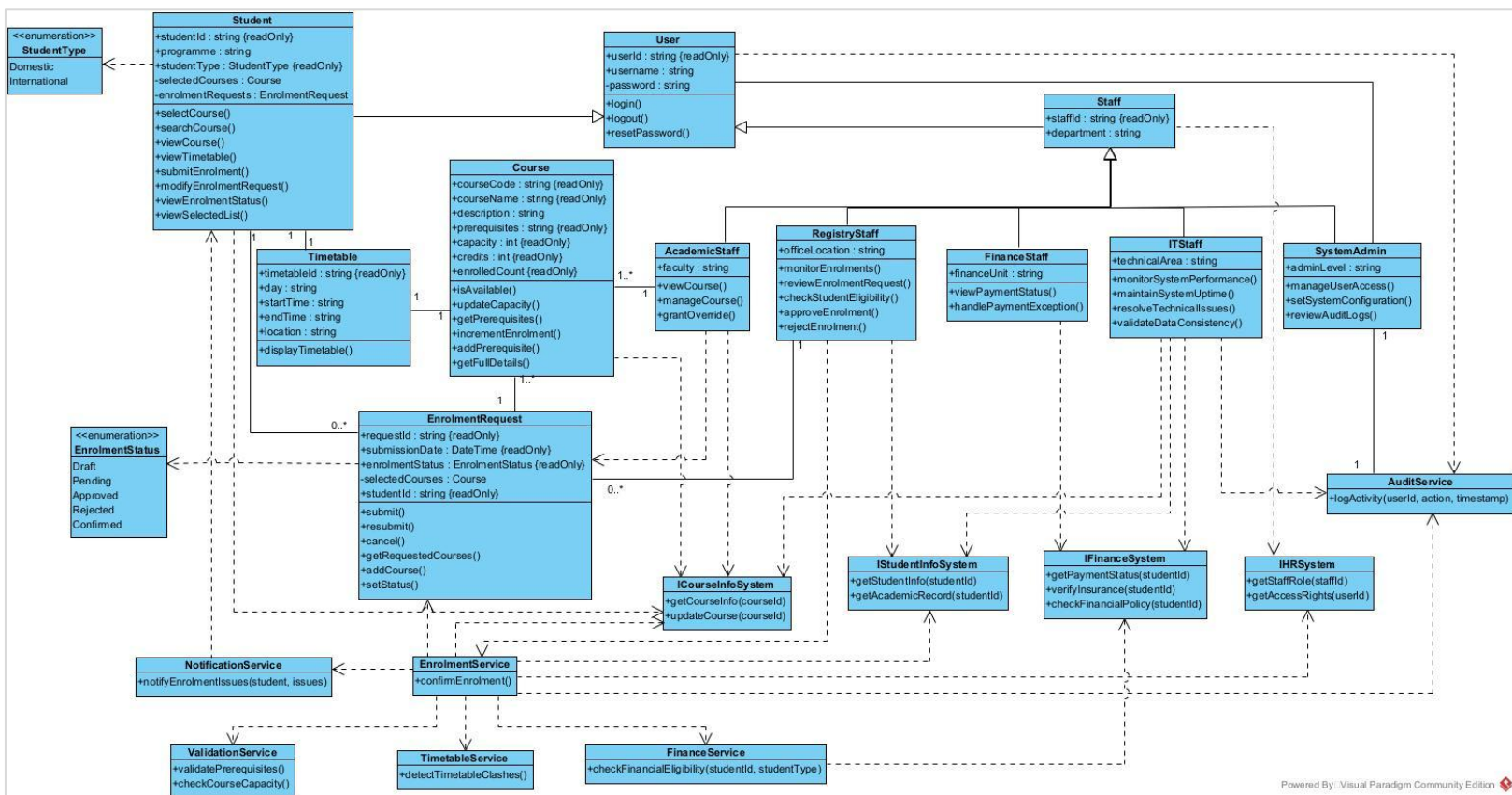


Table 12: Course Enrolment System's Class Diagram

5.2. Mapping to C# classes

Based on the defined class diagram, a direct and consistent mapping to C# classes is developed where each class is implemented as a C# class. Attributes are mapped to properties with appropriate access modifiers (+ public, - private, read-only as { get; private set; }), and operations are implemented as public method signatures without logic. Inheritance represents generalisation relationships, while interfaces are used for external system dependencies to ensure integration and loose coupling. The corresponding C# class structure is shown below.

- **StudentType**

```
public enum StudentType
{
    Domestic,
    International
}
```

- **EnrolmentStatus**

```
public enum EnrolmentStatus
{
    Draft,
    Pending,
    Approved,
    Rejected,
    Confirmed
}
```

- **User**

```
public abstract class User
{
    public string UserId {get; private set;}
    public string Username {get; set;}
    private string _password;

    public void Login() { }
    public void Logout() { }
    public void ResetPassword() { }
}
```

- **Student**

```
public class Student : User
{
    public string StudentId {get; private set;}
    public string Programme {get; set;}
    public StudentType StudentType {get; private set;}
    private List<Course> _selectedCourses = new List<Course>();
    private List<EnrolmentRequest> _enrolmentRequests = new
        List<EnrolmentRequest>();
    public void SearchCourses() { }
    public void SelectCourses() { }
    public void ViewCourses() { }
    public void ViewEnrolledList () { }
    public void ViewTimetable() { }
    public void SubmitEnrolment() { }
    public void ModifyEnrolmentRequest() { }
    public void ViewEnrolmentStatus() { }
}
```

- **Staff**

```
public abstract class Staff : User
{
    public string StaffId {get; private set;}
    public string Department {get; set;}
}
```
- **AcademicStaff**

```
public class AcademicStaff : Staff
{
    public string Faculty {get; set;}

    public void ViewCourse() { }
    public void ManageCourse() { }
    public void GrantOverride() { }
}
```
- **RegistryStaff**

```
public class RegistryStaff : Staff
{
    public string OfficeLocation {get; set;}

    public void MonitorEnrolments() { }
    public void ReviewEnrolmentRequest() { }
    public void CheckStudentEligibility() { }
    public void ApproveEnrolment() { }
    public void RejectEnrolment() { }
}
```
- **FinanceStaff**

```
public class FinanceStaff : Staff
{
    public string FinanceUnit {get; set;}

    public void ViewPaymentStatus() { }
    public void HandlePaymentException() { }
}
```
- **ITStaff**

```
public class ITStaff : Staff
{
    public string TechnicalArea {get; set;}

    public void MonitorSystemPerformance() { }
    public void MaintainSystemUptime() { }
    public void ResolveTechnicalIssues() { }
    public void ValidateDataConsistency() { }
}
```
- **SystemAdmin**

```
public class SystemAdmin : Staff
{
    public string AdminLevel {get; set;}

    public void ManageUserAccess() { }
    public void SetSystemConfiguration() { }
    public void ReviewAuditLogs() { }
}
```
- **Course**

```
public class Course
```



```

{
    public string CourseCode {get; private set;}
    public string CourseName {get; private set;}
    public string Description {get; set;}
    public List<string> Prerequisites {get; private set;}
    public int Capacity {get; private set;}
    public int Credits {get; private set;}
    public int EnrolledCount {get; private set;}

    public bool IsAvailable() {return true;}
    public void UpdateCapacity(int value) { }
    public void IncrementEnrolment() { }
    public void AddPrerequisite(string scourseCode) { }
    public void GetFullDetails() { }
    public List<string> GetPrerequisites() {return Prerequisites;}
}

```

- **Timetable**

```

public class Timetable
{
    public string TimetableId {get; private set;}
    public string Day {get; set;}
    public string StartTime {get; set;}
    public string EndTime {get; set;}
    public string Location {get; set;}

    public void DisplayTimetable() { }
}

```

- **EnrolmentRequest**

```

public class EnrolmentRequest
{
    public string RequestId {get; private set;}
    public string StudentId {get; private set;}
    public DateTime SubmissionDate {get; private set;}
    public EnrolmentStatus EnrolmentStatus {get; private set;}
    private List<Course> _selectedCourses = new List<Course>();

    public void Submit() { }
    public void GetRequestedCourses() { }
    public void AddCourse() { }
    public void SetStatus() { }
    public void Resubmit() { }
    public void Cancel() { }
}

```

- **EnrolmentService**

```

public class EnrolmentService
{
    public void ConfirmEnrolment() { }
}

```

- **ValidationService**

```

public class ValidationService
{
    public void ValidatePrerequisites() { }
    public void CheckCourseCapacity() { }
}

```

- **TimetableService**

```

public class TimetableService
{

```

- ```

 public void DetectTimetableClashes() { }
 }

```
- **FinanceService**

```

public class FinanceService
{
 public void CheckFinancialEligibility(string studentId, StudentType
 type) { }
}

```
  - **NotificationService**

```

public class NotificationService
{
 public void NotifyEnrolmentIssues(Student student, string issues) { }
}

```
  - **AuditService**

```

public class AuditService
{
 public void LogActivity(string userId, string action, DateTime
 timestamp) { }
}

```
  - **ICourseInfoSystem**

```

public interface ICourseInfoSystem
{
 void GetCourseInfo(string courseId);
 void UpdateCourse(string courseId);
}

```
  - **IStudentInfoSystem**

```

public interface IStudentInfoSystem
{
 void GetStudentInfo(string studentId);
 void GetAcademicRecord(string studentId);
}

```
  - **IFinanceSystem**

```

public interface IFinanceSystem
{
 void GetPaymentStatus(string studentId);
 void VerifyInsurance(string studentId);
 void CheckFinancialPolicy(string studentId);
}

```
  - **IHRSystem**

```

public interface IHRSystem
{
 void GetStaffRole(string staffId);
 void GetAccessRights(string userId);
}

```

### 5.3. Anticipated design weaknesses and limitations

Although the designed classes are clear and aligned with the system requirements, there are several potential weaknesses and limitations that should be considered.

Firstly, the system depends heavily on external systems such as ICourseInformationSystem, IStudentInformationSystem, IFinanceSystem, and

IHRSystem. This creates a dependency risk, as any failure, delay, or inconsistency from these systems may affect the enrolment process and overall system reliability.

Secondly, some attributes such as prerequisites and timetable details are represented using simple data types (e.g., String), which may limit the system's ability to handle more complex academic rules or future scalability requirements.

In addition, there is a risk of tight coupling between service classes and other components, which may reduce system flexibility and make future changes more difficult. Error handling and exception management are also not explicitly defined in the design, which may affect system robustness.

Finally, performance risks may occur during peak enrolment periods, as the system requires multiple validation processes and interactions with external systems. Without optimisation, this may impact response time and overall user experience.

## 6. C# implementation

After mapping the class design to C#, the key classes, including User, Student, Staff, RegistryStaff, Course, and EnrolmentRequest, have been fully implemented to support a small working demo of the system. These core classes demonstrate the main structure and behaviour of the enrolment process, following object-oriented principles such as encapsulation and inheritance. The demo program shows how these classes interact to perform essential functions like submitting and monitoring enrolment requests. Other supporting classes and additional features are not fully implemented at this stage and will be completed in further development to extend the system's functionality.

- **User**

```
using System;

public abstract class User
{
 public string UserId { get; private set; }
 public string Username { get; set; }
 private string _password;

 public User(string userId, string username, string password)
 {
 UserId = userId;
 Username = username;
 _password = password;
 }

 public void Login(string username, string password, AuditService audit)
 {
 if (Username == username && _password == password)
 {
 audit.LogActivity(this.UserId, "Successful login", DateTime.Now);
 Console.WriteLine($"{Username} logged in successfully.");
 }
 else
 {
 audit.LogActivity(this.UserId ?? username, "Failed login",
 DateTime.Now);
 Console.WriteLine("Login failed. Invalid credentials.");
 }
 }

 public void Logout(AuditService audit)
 {
 audit.LogActivity(this.UserId, "User logout", DateTime.Now);
 Console.WriteLine($"Session ended for {Username}.");
 Console.WriteLine($"You have been successfully logged out.");
 }
}
```

```

 public void ResetPassword(string oldPassword, string newPassword)
 {
 if (_password == oldPassword)
 {
 _password = newPassword;
 Console.WriteLine("Password updated successfully.");
 }
 else
 {
 Console.WriteLine("Verification failed. Password not changed.");
 }
 }
 }
}

```

- **Student**

```

using System;
using System.Collections.Generic;

public class Student : User
{
 public string StudentId { get; private set; }
 public string Programme { get; set; }
 public StudentType StudentType { get; private set; }
 private List<Course> _selectedCourses = new List<Course>();
 private List<EnrolmentRequest> _enrolmentRequests = new
List<EnrolmentRequest>();

 public Student(string userId, string username, string password, string
studentId, string programme, StudentType type)
 : base(userId, username, password)
 {
 this.StudentId = studentId;
 this.Programme = programme;
 this.StudentType = type;
 }

 public void SearchCourses(ICourseInfoSystem cis)
 {
 Console.WriteLine($"--- Searching AUT database for {this.Programme} ---
");
 cis.GetCourseInfo("ALL");
 }
 public void SelectCourses(Course course)
 {
 if (course.IsAvailable())
 {
 _selectedCourses.Add(course);
 Console.WriteLine($"Course {course.CourseCode} added to your
selection.");
 }
 else
 {
 Console.WriteLine($"Cannot select {course.CourseCode}: Course is at
full capacity.");
 }
 }
 public void ViewCourses(Course course)
 {
 Console.WriteLine("--- Course details (Dropdown) ---");
 Console.WriteLine($"Code: {course.CourseCode}");
 }
}

```

```

 Console.WriteLine($"Name: {course.CourseName}");
 Console.WriteLine($"Description: {course.Description}");
 Console.WriteLine($"Credits: {course.Credits}");
 Console.WriteLine($"Current Capacity: {course.Capacity}");

 Console.Write("Prerequisites: ");
 if (course.Prerequisites.Count == 0) Console.Write("None");
 else Console.Write(string.Join(", ", course.Prerequisites));
 Console.WriteLine("\n-----");
 }
 public void ViewEnrolledList()
 {
 Console.WriteLine("--- Selected courses ---");

 if (_selectedCourses.Count == 0)
 {
 Console.WriteLine("No courses selected yet.");
 }
 else
 {
 foreach (var course in _selectedCourses)
 {
 Console.WriteLine($"> {course.CourseCode} - {course.CourseName}
({course.Credits} Credits)");
 }
 Console.WriteLine("\n-----");
 }
 }
 public void ViewTimetable(Timetable timetable)
 {
 Console.WriteLine($"Generating timetable for {StudentId}...");
 timetable.DisplayTimetable();
 }

 public void SubmitEnrolment(AuditService audit)
 {
 if (_selectedCourses.Count == 0)
 {
 Console.WriteLine("Submission failed: No courses selected.");
 return;
 }

 string newRequestId = "REQ-" + Guid.NewGuid().ToString().Substring(0,
5).ToUpper();
 EnrolmentRequest newRequest = new EnrolmentRequest(newRequestId,
StudentId, _selectedCourses);

 newRequest.Submit();

 _enrolmentRequests.Add(newRequest);
 _selectedCourses.Clear();

 audit.LogActivity(this.UserId, $"Submitted Enrolment {newRequestId}",
DateTime.Now);

 Console.WriteLine($"Enrolment request {newRequestId} has been submitted
successfully.");
 }

 public void ModifyEnrolmentRequest(string requestId)
 {

```

```

 var request = _enrolmentRequests.Find(r => r.RequestId == requestId);

 if (request != null && request.EnrolmentStatus ==
EnrolmentStatus.Pending)
 {
 request.Cancel();
 Console.WriteLine($"Request {requestId} has been pulled back for
modification.");
 }
 else
 {
 Console.WriteLine("Modification error: Request not found or already
processed by registry staff.");
 }
 }

 public void ViewEnrolmentStatus()
 {
 Console.WriteLine($"--- Enrolment history for {this.Username} ---");
 if (_enrolmentRequests.Count == 0)
 {
 Console.WriteLine("No enrolment requests found.");
 }
 else
 {
 foreach (var req in _enrolmentRequests)
 {
 Console.WriteLine($"Request ID: {req.RequestId} | Submitted:
{req.SubmissionDate.ToShortDateString()} | Status: {req.EnrolmentStatus}");
 }
 }
 Console.WriteLine("\n-----");
 }
}

```

- **Staff**

```

public abstract class Staff : User
{
 public string StaffId { get; private set; }
 public string Department { get; set; }

 public Staff(string userId, string username, string password, string
staffId, string dept)
 : base(userId, username, password)
 {
 this.StaffId = staffId;
 this.Department = dept;
 }
}

```

- **RegistryStaff**

```

using System;
using System.Collections.Generic;
using System.Linq;

public class RegistryStaff : Staff
{
 public string OfficeLocation { get; set; }
}

```

```

 public RegistryStaff(string userId, string username, string password, string
staffId, string dept, string office)
 : base(userId, username, password, staffId, dept)
 {
 this.OfficeLocation = office;
 }

 public void MonitorEnrolments(List<EnrolmentRequest> allRequests)
 {
 Console.WriteLine($"--- Enrolment monitoring ---");

 int pendingCount = 0;
 foreach (var req in allRequests)
 {
 if (req.EnrolmentStatus == EnrolmentStatus.Pending) pendingCount++;
 }

 Console.WriteLine($"Total requests: {allRequests.Count}");
 Console.WriteLine($"Action required (pending): {pendingCount}");
 Console.WriteLine("\n--- Enrolment request list ---");

 foreach (var req in allRequests)
 {
 var courseCodes = string.Join(", ",
req.GetRequestedCourses().Select(c => c.CourseCode));

 Console.WriteLine($"Request ID: {req.RequestId}, " +
 $"Student: {req.StudentId}, " +
 $"Courses: [{courseCodes}], " +
 $"Status: {req.EnrolmentStatus}");
 }
 Console.WriteLine("\n-----");
 }

 public void ReviewEnrolmentRequest(Student student, EnrolmentRequest
request)
 {
 Console.WriteLine($"Staff {this.Username} is reviewing Request:
{request.RequestId}");
 Console.WriteLine($"Student: {student.Username} ({student.StudentId})");
 Console.WriteLine($"Programme: {student.Programme}");
 Console.WriteLine($"Submission Date:
{request.SubmissionDate.ToLongDateString()}");
 Console.WriteLine($"Current Status: {request.EnrolmentStatus}");

 Console.WriteLine("System Note: Student has met basic financial
eligibility.");
 Console.WriteLine("\n-----");
 }

 public bool CheckStudentEligibility(Student student, IStudentInfoSystem sis,
IFinanceSystem fis)
 {
 Console.WriteLine($"Running verification for: {student.Username}
({student.StudentId})");

 Console.WriteLine("Step 1: Fetching academic records from SIS...");
 sis.GetAcademicRecord(student.StudentId);

 Console.WriteLine("Step 2: Verifying outstanding fees and
insurance...");
 }
 }
}

```



```

 fis.GetPaymentStatus(student.StudentId);
 fis.VerifyInsurance(student.StudentId);

 bool isEligible = true;

 if (isEligible)
 {
 Console.WriteLine("Student is eligibility for enrolment.");
 return true;
 }
 else
 {
 Console.WriteLine("Student is ineligibility due to outstanding requirements.");
 return false;
 }
 }

 public void ApproveEnrolment(EnrolmentRequest request, AuditService audit)
 {
 if (request.EnrolmentStatus == EnrolmentStatus.Pending)
 {
 request.SetStatus(EnrolmentStatus.Approved);

 audit.LogActivity(this.UserId, $"Approved enrolment {request.RequestId}", DateTime.Now);

 Console.WriteLine($"Request {request.RequestId} has been approved by {this.Username}.");
 }
 else
 {
 Console.WriteLine($"Cannot approve: Request is in {request.EnrolmentStatus} state.");
 }
 }

 public void RejectEnrolment(EnrolmentRequest request, string reason, AuditService audit)
 {
 request.SetStatus(EnrolmentStatus.Rejected);

 audit.LogActivity(this.UserId, $"Rejected enrolment {request.RequestId}. Reason: {reason}", DateTime.Now);

 Console.WriteLine($"Request {request.RequestId} has been rejected. Reason: {reason}");
 }
}

```

- **EnrolmentRequest**

```

using System;
using System.Collections.Generic;

public class EnrolmentRequest
{
 public string RequestId { get; private set; }
 public string StudentId { get; private set; }
 public DateTime SubmissionDate { get; private set; }
 public EnrolmentStatus EnrolmentStatus { get; private set; }
 private List<Course> _requestedCourses = new List<Course>();
}

```

```

 public EnrolmentRequest(string requestId, string studentId, List<Course>
requestedCourses)
 {
 this.RequestId = requestId;
 this.StudentId = studentId;
 this._requestedCourses.AddRange(requestedCourses);
 this.SubmissionDate = DateTime.Now;
 this.EnrolmentStatus = EnrolmentStatus.Draft;
 }

 public void Submit()
 {
 if (EnrolmentStatus == EnrolmentStatus.Draft)
 {
 EnrolmentStatus = EnrolmentStatus.Pending;
 this.SubmissionDate = DateTime.Now;
 }
 }

 public void SetStatus(EnrolmentStatus newStatus)
 {
 this.EnrolmentStatus = newStatus;
 }

 public void Cancel()
 {
 if (EnrolmentStatus == EnrolmentStatus.Pending || EnrolmentStatus ==
EnrolmentStatus.Draft)
 {
 EnrolmentStatus = EnrolmentStatus.Cancelled;
 Console.WriteLine($"Request {RequestId} has been cancelled.");
 }
 }

 public void AddCourse(Course course)
 {
 if (!_requestedCourses.Contains(course))
 {
 _requestedCourses.Add(course);
 }
 }

 public List<Course> GetRequestedCourses()
 {
 return _requestedCourses;
 }

 public void Resubmit()
 {
 if (this.EnrolmentStatus == EnrolmentStatus.Rejected ||
this.EnrolmentStatus == EnrolmentStatus.Cancelled)
 {
 this.EnrolmentStatus = EnrolmentStatus.Pending;

 this.SubmissionDate = DateTime.Now;

 Console.WriteLine($"Request {RequestId} has been resubmitted for
review.");
 }
 else
 {
 Console.WriteLine($"Cannot resubmit. Current status is:
{EnrolmentStatus}");
 }
 }
}

```

}  
• **Course**

```
using System;
using System.Collections.Generic;

public class Course
{
 public string CourseCode { get; private set; }
 public string CourseName { get; private set; }
 public string Description { get; set; }
 public int Credits { get; private set; }
 public int Capacity { get; private set; }
 public int EnrolledCount { get; private set; }
 public List<string> Prerequisites { get; private set; }

 public Course(string code, string name, int credits, int capacity)
 {
 this.CourseCode = code;
 this.CourseName = name;
 this.Credits = credits;
 this.Capacity = capacity;
 this.EnrolledCount = 0;
 this.Prerequisites = new List<string>();
 }

 public void UpdateCapacity(int value)
 {
 if (value >= EnrolledCount)
 {
 this.Capacity = value;
 Console.WriteLine($"{CourseCode} capacity updated to {value}.");
 }
 else
 {
 Console.WriteLine($"Update denied for {CourseCode}:");
 Console.WriteLine($" - Requested capacity: {value}");
 Console.WriteLine($" - Current enrolments: {EnrolledCount}");
 }
 }

 public List<string> GetPrerequisites() { return Prerequisites; }
 public bool IsAvailable()
 {
 return EnrolledCount < Capacity;
 }

 public void IncrementEnrolment()
 {
 if (IsAvailable())
 {
 EnrolledCount++;
 }
 }

 public void AddPrerequisite(string courseCode)
 {
 if (!Prerequisites.Contains(courseCode))
 {
 Prerequisites.Add(courseCode);
 }
 }

 public string GetFullDetails()
 {

```

```

 string prereqs = Prerequisites.Count > 0 ? string.Join(", ",
Prerequisites) : "None";
 return $"{CourseCode}: {CourseName}\n" +
 $"Credits: {Credits}\n" +
 $"Prerequisites: {prereqs}\n" +
 $"Availability: {Capacity - EnrolledCount} spots remaining.";
 }
}

```

- **Demo program**

```

using System;
using System.Collections.Generic;

class Program
{
 static void Main(string[] args)
 {
 AuditService audit = new AuditService();

 // Create a course and add a prerequisite manually
 Course ense706 = new Course("ENSE706", "Data Process and Software
Modelling", 15, 20);
 ense706.Description = "This course introduces the Unified Modeling
Language (UML) as a key tool for requirements analysis, object-oriented software
design, and identifying use cases.";
 ense706.AddPrerequisite("ENSE602");

 // Create a student and staff
 Student student = new Student("U001", "AdamTran", "pass123", "12345678",
"BCIS", StudentType.Domestic);
 RegistryStaff staff = new RegistryStaff("ST01", "Admin_Sarah",
"admin789", "1234567", "Registry", "WZ321");

 // A list to simulate the "Database" of all system requests
 List<EnrolmentRequest> systemRequests = new List<EnrolmentRequest>();

 Console.WriteLine("=== AUT ENROLMENT SYSTEM START ===\n");

 // Student action
 student.Login("AdamTran", "pass123", audit);

 // Simulate searching and viewing
 student.ViewCourses(ense706);

 Console.WriteLine("\nStudent is selecting COMP603...");
 student.SelectCourses(ense706);
 student.ViewEnrolledList();

 // Submit the enrolment
 student.SubmitEnrolment(audit);

 // Create a request
 var demoReq = new EnrolmentRequest("REQ-ABC12", student.StudentId, new
List<Course> { ense706 });
 demoReq.Submit(); // Set to Pending
 systemRequests.Add(demoReq);

 // Registry staff action
 Console.WriteLine("\nSwitching to registry staff view...");
 staff.Login("Admin_Sarah", "admin789", audit);
 }
}

```

```
 // Monitor the dashboard
 staff.MonitorEnrolments(systemRequests);

 // Review the specific request
 staff.ReviewEnrolmentRequest(student, demoReq);

 // Final Approval
 staff.ApproveEnrolment(demoReq, audit);

 // Check history
 student.ViewEnrolmentStatus();

 Console.WriteLine("\n=== DEMO END ===");
 student.Logout(audit);
 }
}
```

## 7. Conclusion

In conclusion, the Student Enrolment System has been analysed, designed, and partially implemented based on the given requirements. UML diagrams were used to clearly model the system structure and behaviour, which were then mapped into a C# implementation. The key classes were successfully developed to demonstrate the core functionality of the system, including enrolment request handling and monitoring. Future work will focus on completing the remaining classes and extending the system to fully meet all functional and non-functional requirements.

## References

- AUT (2018). City Campus' newest building now open.  
<https://www.aut.ac.nz/news/stories/city-campus-newest-building-now-open>
- AUT (n.d.-a). *Facts and figures about AUT*. <https://www.aut.ac.nz/about/auts-leadership/fact-and-figures-about-aut>
- AUT (n.d.-b). *AUT faculties and schools*. <https://www.aut.ac.nz/about/faculties-and-schools>
- AUT (n.d.-c). *Student services online*. <https://www.aut.ac.nz/student-life/studying/student-services-online>
- AUT (n.d.-d). *Enrolling in courses and classes*. <https://www.aut.ac.nz/student-life/studying/enrolling-in-courses-and-classes>
- AUT (n.d.-e). *IT services*. <https://www.aut.ac.nz/student-life/studying/it-services>
- AUT (n.d.-f). *Financial services and support for students*. <https://www.aut.ac.nz/student-life/support-services/student-hub/financial-support>
- ISO. (2023). *ISO/IEC 25010:2023 – Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*.  
<https://www.iso.org/obp/ui/en/#iso:std:iso-iec:25010:ed-2:v1:en:term:3.2.3>